

Distance Sensor

Hardware Overview

What is it?

The **Ultrasonic Sensor** (model HC-SR04) emits a sound pulse that is too high for humans to hear (ultrasound) and listens for the echo.

Electrical Explanation

The sensor has two main parts that look like eyes:

1. **Transmitter (Trig):** Acts like a speaker. It sends a short burst of ultrasonic sound.
2. **Receiver (Echo):** It registers the sound bouncing back.

How it measures distance: K-Comp measures the **Time of Flight** - how long it takes for the sound to travel to the object and return. Since we know the speed of sound in air (approx. 343 m/s), we can calculate the distance: $\text{Distance} = (\text{Time} \times \text{Speed of Sound}) / 2$.

Key Hardware Facts:

- **Trigger Pin:** Connected to **Digital Pin D11**.
- **Echo Pin:** Connected to **Digital Pin D10**.
- **Range:** It works best between **2 cm** and **400 cm**.

Signal Flow: Processor → Sensor sends sound → Sound hits object → Echo returns → Sensor signals Processor.

Software & Programming

kcomp_distance.h

Handling the microsecond timing manually is tricky, so we use a library to do the math for us.

Usage: `#include <kcomp_distance.h>`.

Function Reference

Function Name	Description & Parameters
<code>void initDistanceSensor()</code>	Setup: Configures the Trigger pin as Output and Echo pin as Input. This must be called once inside <code>setup()</code> .
<code>int getDistanceMM()</code>	Measure: Triggers a pulse, waits for the echo, and calculates the distance. Returns: The distance to the object in millimeters (mm) as an integer.

Pitfall: "Ghost Echoes"

Ultrasonic sensors are not perfect.

1. **Soft Objects:** Fabric or fur might absorb the sound, returning no echo.
2. **Angled Surfaces:** If the object is at a steep angle, the sound might bounce *away* from the sensor instead of back to it.

3. **Delays:** Sound is slow! If you try to measure 1000 times a second, the old echoes might confuse the new measurement. Always add a small `delay()` between readings.



Sample Program

This program measures the distance and prints it to the Serial Monitor in millimeters.

```
#include <kcomp_distance.h> // 1. Include the library

void setup() {
  Serial.begin(115200);
  initDistanceSensor(); // 2. Initialize the sensor hardware
  Serial.println("Sensor Ready!");
}

void loop() {
  // 3. Get the distance in millimeters
  int distMM = getDistanceMM(); //

  // 4. Print the result
  Serial.print("Object is ");
  Serial.print(distMM);
  Serial.println(" mm away.");

  // 5. Wait a bit (important for sound waves to clear!)
  delay(500); //
}
```



Try it out: You can find the full sample program **SimpleDistanceSensor** in the K-Comp examples folder!